



PY#1 – Sistemas Operativos

Profesor: Christian Campos

Estudiantes:

Julián Lizano Monge - 2024188887

Instituto Tecnológico de Costa Rica

Fecha de entrega: 6/10/2026

1. Introducción

Este documento describe el diseño del sistema operativo simulado: cómo se organizó en paquetes, qué responsabilidad tiene cada uno y cómo se comunican entre sí. El sistema recrea una minicomputadora con recursos de memoria principal, almacenamiento secundario, dispositivo de E/S (pantalla y teclado) y CPU, capaz de ejecutar programas escritos en un mini-ensamblador (.asm).

El diseño sigue los principios de Stallings (Operating Systems: Internals and Design Principles, 9ª edición), aplicando:

- Modelo de 7 estados de proceso (nuevo, preparado, ejecución, bloqueado, suspendido-preparado, suspendido-bloqueado y terminado).
- Cambio de contexto (sección 3.4).
- Tabla de memoria y tabla de archivos (sección 3.3).
- Particionamiento dinámico con estrategia first-fit (sección 7.2).

El objetivo es que el estudiante comprenda cómo un Sistema Operativo orquesta los recursos internos para ejecutar uno o varios programas, garantizando que las peticiones del usuario sean atendidas de forma eficiente.

2. Diagrama de paquetes



El sistema se organiza en cinco capas con dependencias unidireccionales. Cada capa solo conoce a las capas inferiores, lo que facilita el mantenimiento y la sustitución de componentes.

El sistema se organiza en capas: presentación (gui), lógica del sistema operativo (logica), modelo del hardware (modelo) y configuración (config).

3. Descripción de los paquetes

Paquete	Responsabilidad	Clases principales
App	Punto de entrada. Crea y muestra la ventana principal.	Main
gui	Interfaz gráfica. Muestra procesos, memoria, disco y pantalla, y recibe las acciones del usuario (cargar, ejecutar, paso a paso, configurar, estadísticas).	VentanaPrincipal, PanelProcesos, PanelMemoria, PanelDisco, PanelPantalla, VentanaConfiguracion, VentanaEstadisticas, Paleta
logica	Núcleo del S.O.: carga de programas, ensamblado, planificación, despacho, ejecución de instrucciones, interrupciones y asignación de memoria.	GestorProcesos, Ensamblador, Despachador, EjecutorCPU, Interrupciones, ListaProcesos, ListaDeTrabajos, ParticionadorDinamico, Planificador
logica.planificacion	Algoritmos de planificación intercambiables mediante una interfaz.	EstrategiaPlanificacion, PlanificadorFCFS
modelo	Representación de los componentes del hardware y las estructuras de datos.	Memoria, CPU, Disco, BCP, Instruccion, EstadoProceso
config	Lectura, validación y guardado de config.txt, con corrección automática de valores inválidos.	ConfiguracionExterna

4. Dependencias entre paquetes

- App depende solo de gui.
- gui depende de logica (para operar el sistema), de modelo (para mostrar el estado) y de config (para leer y guardar la configuración).
- logica depende de modelo y delega la política de planificación a logica.planificacion.
- modelo no depende de ningún otro paquete, lo que lo hace la base del sistema.

5. Decisiones de diseño

- **Separación en capas:** la interfaz no contiene reglas del S.O.; solo invoca al GestorProcesos. Así se puede cambiar la interfaz sin tocar la lógica.
- **Patrón Strategy en la planificación:** Planificador usa una EstrategiaPlanificacion. Hoy se usa FCFS, pero se pueden agregar otras (Round Robin, SJF) sin modificar el resto.
- **Memoria dividida en kernel y usuario:** el kernel guarda los BCP y el espacio de usuario guarda los procesos, gestionado con particionado dinámico (ParticionadorDinamico).
- **Disco con tres zonas:** índice de archivos, área de swap y área de archivos. El swap permite suspender procesos (READY_SUSPEND, BLOCKED_SUSPEND) cuando no hay espacio en memoria.
- **Cola de trabajos:** si un programa no cabe en memoria, queda en ListaDeTrabajos hasta que haya hueco.
- **Interrupciones:** Interrupciones atiende teclado, pantalla y operaciones de archivos, e indica si el proceso sigue corriendo, se bloquea o termina.
- **Comunicación por callbacks:** el GestorProcesos notifica a la interfaz (salida en pantalla, solicitud de teclado, operaciones de archivo) sin depender de las clases de gui.
- **Configuración externa validada:** config.txt permite cambiar tamaños de memoria, disco, kernel y swap, y se corrige solo si hay valores inválidos.
- **DX como String:** el registro DX almacena el nombre del archivo como cadena de texto para INT 21H (crear, abrir, leer, escribir, eliminar), siguiendo la indicación del profesor.

6. Flujo general de ejecución

El ciclo de vida de un proceso en el sistema pasa por seis etapas bien definidas, desde que el usuario carga un archivo .asm hasta que el proceso termina y se generan sus estadísticas.

Etapas 1: Carga del archivo

El usuario presiona el botón "Cargar archivos" y selecciona uno o varios archivos .asm desde el disco. Cada archivo pasa al Ensamblador, que realiza una validación completa antes de continuar.

Validaciones aplicadas:

- El archivo existe, es legible y tiene extensión .asm.
- Cada línea tiene un opcode conocido y un número válido de argumentos.
- Los registros y los números (decimales o hexadecimales) son válidos.
- El programa contiene al menos una instrucción INT 20H para finalizar correctamente.

Si alguna validación falla, se muestra un mensaje claro con la línea exacta del error y el archivo se descarta.

Etapas 2: Ensamblado

Si el archivo pasa las validaciones, el Ensamblador traduce cada línea a un objeto Instrucción con:

- El opcode (LOAD, MOV, ADD, etc.).
- La lista de argumentos ya normalizados.
- El peso asociado a la instrucción (número de ciclos que consumirá).
- El resultado es una lista ordenada de instrucciones lista para ejecutarse.

Etapas 3: Asignación de recursos

El GestorProcesos intenta asignar los recursos necesarios para el nuevo proceso:

1. Partición de memoria (first-fit)

El ParticionadorDinamico busca el primer hueco libre en la zona usuario que pueda contener las instrucciones. Si lo encuentra, marca el bloque como ocupado y lo registra en la TablaMemoria del kernel.

2. BCP en el kernel

Si hay un BCP libre, se crea un nuevo Bloque de Control de Proceso con estado READY y se inicializan todos sus campos (registros, pila, tiempos, etc.).

3. Casos alternativos

- Si no hay partición disponible, el proceso va a la ListaDeTrabajos (en RAM) con estado NEW.
- Si tampoco hay espacio en la ListaDeTrabajos, el proceso se envía al swap del disco con estado READY_SUSPEND y se registra en el índice ASM con zona VIRTUAL.
- Si no hay espacio en swap, se reporta un error y el proceso no se carga.

Etapas 4: Planificación y despacho

Cuando la CPU queda libre, el Planificador selecciona el siguiente proceso según la política activa (FCFS por defecto): el primero en llegar es el primero en ejecutarse.

- El Despachador realiza el cambio de contexto siguiendo los pasos de Stallings:
- Guarda el contexto del proceso saliente (registros de CPU → BCP).
- Actualiza el estado del BCP saliente (READY, BLOCKED, etc.).
- Selecciona el siguiente BCP de la ListaProcesos.
- Restaura el contexto del proceso entrante (BCP → registros de CPU).
- Marca el proceso como RUNNING y crea un EjecutorCPU para él.

Etapas 5: Ejecución de instrucciones

El EjecutorCPU ejecuta el ciclo fetch-decode-execute:

Fetch: Lee la instrucción apuntada por el PC desde la memoria.

Decode: Identifica el opcode y sus argumentos.

Execute: Realiza la operación correspondiente.

Cada instrucción tiene un peso (número de ciclos). El proceso consume su peso a lo largo de varios pasos de CPU: un ADD (peso 3) tarda 3 ciclos, un MOV (peso 1) tarda 1 ciclo, etc.

Durante la ejecución pueden ocurrir tres cosas:

El proceso sigue ejecutándose (RUNNING)

- Incrementa el PC y continúa con la siguiente instrucción.

El proceso se bloquea (BLOCKED)

- Ocurre con interrupciones de E/S:
- INT 09H (lectura de teclado): el proceso espera a que el usuario ingrese un valor 0-255.
- INT 21H (manejo de archivos): el proceso espera a que el disco complete la operación.

El proceso pasa a BLOCKED y la CPU puede ejecutar otro proceso.

c) El proceso termina (EXIT)

Ocurre con INT 20H. El proceso pasa a EXIT y se notifica al GestorProcesos.

Etapas 6: Terminación y liberación de recursos

Cuando el proceso termina:

1. Se guarda un snapshot (BCPTerminado) con toda la información relevante: ID, estado final, hora de inicio, hora de fin, base, alcance y prioridad. Este snapshot se usa para las estadísticas.
2. Se liberan los recursos de memoria:
 - a. La partición de usuario se devuelve al ParticionadorDinamico (que fusiona huecos contiguos).
 - b. El bloque de la TablaMemoria se marca como libre.
 - c. El BCP se libera en el kernel.
3. Se reactiva el siguiente proceso:
 - a. El GestorProcesos verifica si hay procesos en la ListaDeTrabajos. Si es así, el primero sale de la lista y se intenta admitir en RAM:
 - b. Se lee el bloque de swap del disco.
 - c. Se copia el contenido a la zona de archivos del disco (respaldo).
 - d. Se elimina la entrada del índice ASM (ya no está solo en swap).
 - e. Se libera el bloque de swap.
 - f. Se crea un nuevo BCP y se admite en RAM.
4. Se generan las estadísticas finales:
 - a. Al finalizar toda la ejecución, la GUI muestra un resumen en el panel de pantalla y ofrece una ventana modal (VentanaEstadisticas) con la tabla detallada de procesos terminados: ID, hora de inicio, hora de fin, duración en segundos y porcentaje del tiempo total.

7. Conclusión

El sistema implementa un gestor de procesos completo sobre una arquitectura en capas, con separación estricta entre interfaz, lógica, modelo y configuración. Los principios de Stallings se aplican en el modelo de 7 estados, la tabla de memoria, la tabla de archivos y el cambio de contexto. La planificación es intercambiable (patrón Strategy), la comunicación con la GUI es desacoplada (Observer/callbacks) y la seguridad se garantiza con validación estricta y protección de memoria.

El diseño cumple con todos los requisitos del proyecto: GUI completa, dos modos de ejecución, validación de archivos, ejecución de instrucciones con pesos, interrupciones, memoria virtual, planificación FCFS, estadísticas y estrategia de seguridad.